When Prediction Isn't Enough: Decision-Criteria-Informed Learning for Threshold-Based Decisions

Franklin Doane
Dept. of Computer Science
Tennessee Technological University
Cookeville, United States
doanefranklin89@gmail.com

Jesse Roberts
Dept. of Computer Science
Tennessee Technological University
Cookeville, United States
jtroberts@tntech.edu

***Abstract*—A good predictor doesn't necessarily make a good decider. In threshold-based settings, where predicted values are compared against decision criteria to select actions, this separation is particularly costly. We propose Decision-Criteria-Informed (DCI) classifiers, which replace the prediction model and threshold policy with a single classifier that receives the decision criteria as direct inputs and is trained to predict the resulting action. In blackjack, a DCI random forest achieves a decisive-outcome win rate of 0.433 against 0.291 for the prediction-focused pipeline, a statistically significant gap despite identical model capacity. To test whether this advantage is consistent across model class and domain, we evaluate LSTMs for short-term stock trading. There, a DCI LSTM outperforms both prediction-focused and decision-aware baselines in precision and profitability ($p \approx 0.001$), showing criteria-awareness rather than decision-awareness alone is the key driver. Because the same state can be a positive (buy) or a negative (non-buy) depending on the threshold, the model learns to be appropriately uncertain rather than overconfident, a capability the prediction-focused pipeline does not possess.**

***Index Terms*—Decision-focused learning, predict-then-optimize, threshold-based policies, decision-criteria-informed classifiers, feature extraction and classification, neural network learning, game playing, forecasting applications**

I. Introduction

Any complex decision can be broken into two stages: gathering and estimating relevant information, and then selecting an action based on what is known. Both stages are shaped by the decision-maker's criteria, the explicit or implicit constraints that define what a good outcome looks like. A consumer shopping for a coffee maker, for instance, must account for counter space and brand preference not just at the moment of purchase, but throughout the entire process. Ignoring these criteria at any stage risks undermining the final decision, as illustrated in Fig. 1.

Two common paradigms address these predict-then-optimize problems. Prediction Focused Learning (PFL) optimizes the stages separately. A predictor is first trained to estimate the relevant unknown information as accurately as possible based on what is known. Next, a decision maker is created separately and optimized using the predictor's output. Decision-Focused Learning (DFL) takes a different approach. The predictor and the decision maker are optimized together [1]. This results in an estimator tailored not necessarily to produce the most accurate prediction, but rather to support the best decision further down the pipeline.

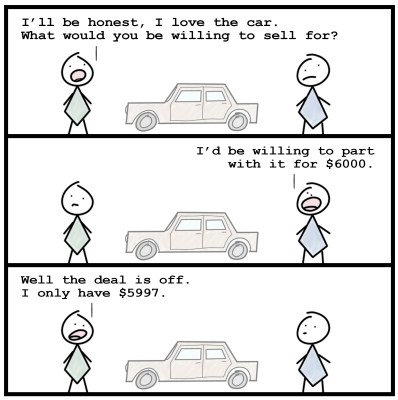


Fig. 1. A comic depicting a character who fails to recognize that their decision criteria should be part of their negotiation.

In this paper, we examine a specific class of predict-then-optimize problems in which decisions are determined by applying thresholds to predicted features. Existing DFL methods often optimize through, or construct surrogate losses for, downstream decision procedures [3]–[5]. In threshold-based settings, we propose a simpler alternative: replacing the predictor and threshold policy with a classifier that receives the decision criteria as inputs and directly predicts the policy's action. We call this model a Decision-Criteria-Informed (DCI) classifier.

We conduct two experiments to test both whether DCI outperforms PFL and why. First, we examine DCI random forests in a simplified game of blackjack, establishing that the approach produces substantial gains over a PFL baseline despite identical model capacity. This result is promising but leaves the source of the advantage unexplained, since the decision criteria in blackjack are intrinsic to the task rather

than user-specified. The subsequent stock-trading experiment is designed to address this directly. By introducing a decision-aware model that does not receive criteria as input, we can isolate the contribution of criteria-awareness from decision-awareness alone. The results show that criteria-awareness calibrates model confidence, which we propose as the key driver of DCI's advantage. We further show that repeated training exposure to identical inputs and targets with varying criteria improves DCI's precision but not downstream performance.

Contributions

1) We introduce DCI classifiers for threshold-based predict-then-optimize settings.
2) We compare DCI models with PFL baselines in blackjack and short-term stock-trading experiments.
3) We conduct an ablation study showing the impact of criteria awareness and the impact of varying and shuffling decision criteria during model training.

II. Related Work

Decision-Focused Learning is an alternative to training predict-then-optimize pipelines in two separate stages. Mandi et al. provide a survey of DFL, and this paper relies on their framing of DFL and PFL [1]. Qi and Shen also provide a tutorial survey of integrated prediction-and-optimization approaches in operations management, emphasizing that data-driven models should be evaluated by downstream decision quality [2].

Several DFL-related approaches have been proposed for predict-then-optimize problems. They can be divided into differentiable and non-differentiable optimization approaches. Differentiable approaches typically involve passing gradients between stages. For example, Priya L. Donti et al. proposed an end-to-end method for learning predictive models under a downstream stochastic optimization objective [3]. Brandon Amos et al. developed a framework for differentiating through Model Predictive Control (MPC), allowing MPC policies to be embedded in end-to-end learning systems [6]. For non-differentiable optimization approaches, Adam N. Elmachtoub and Paul Grigas propose creating a task-specific surrogate loss function that directly optimizes the prediction task based on the quality of the downstream decision [4]. Sanket Shah et al. take it a step further and replace hand-crafted surrogate losses with a learned task-specific loss function [5].

Other related work uses Machine Learning (ML) to predict CO solutions directly. James Kotary et al. discuss this class of methods in their survey of end-to-end CO learning, highlighting direct solution prediction as a way to obtain approximate solutions to CO problems efficiently [7]. Rares Cristian et al. developed neural-network approximators for CO problems that use constraint-enforcing projection mechanisms [8]. Their approach was found to improve computational efficiency while maintaining competitive loss quality.

The proposed methods in this work are related to direct CO-solution prediction because they also replace an intermediate prediction stage with direct decision prediction. However, DCI models do not predict traditional full CO solutions. Instead, we focus on settings where a threshold operation maps predicted features to discrete actions. We also introduce decision labels that are conditioned on decision criteria provided as model inputs. To our knowledge, this is the first work to explore the benefits of this technique.

III. Methodology

A standard predict-then-optimize solution would use a prediction model M_P that maps the known information state $\mathbf{z}$ to predicted features $\hat{\mathbf{c}}$. Next, the CO problem would require a decision maker π to map estimated features $\hat{\mathbf{c}}$ to actions a. Note that the known information $\mathbf{z}$ may also be considered directly by π. Our notation is adapted from the general DFL formulation from the survey by Mandi et al. [1], but we specialize by defining a downstream threshold-based decision function π that selects actions a, rather than discussing a full CO solution that produces optimal actions, for example $x^*(\hat{\mathbf{c}})$. We define a DCI classifier as any classifier trained on both state variables and decision criteria to predict the action induced by a given decision policy assuming perfect knowledge of unknown features.

A threshold-based ruleset for decision making can be expressed as follows. A policy π for selecting actions a is defined as in (1), where the decision criteria matrix T contains a threshold vector that separates each possible action type based on each feature. Unless otherwise stated, vector inequalities are evaluated elementwise, and an action is selected only when all feature-level threshold conditions for that action are satisfied. With n possible actions and m predicted features, $\hat{\mathbf{c}} \in \mathbb{R}^m$ and $T \in \mathbb{R}^{(n-1)\times m}$.

$$\pi(\hat{\mathbf{c}}, T) = \begin{cases} \text{action } 1 & \text{if } \hat{\mathbf{c}} < T_1 \\ \text{action } 2 & \text{if } T_1 < \hat{\mathbf{c}} < T_2 \\ \dots & \\ \text{action } n\text{–}1 & \text{if } T_{n-2} < \hat{\mathbf{c}} < T_{n-1} \\ \text{action } n & \text{if } T_{n-1} < \hat{\mathbf{c}} \end{cases} \tag{1}$$

In this paper, both of the case studies we conduct involve tasks with only two possible choices at each timestep. In such cases, only a single threshold vector $\mathbf{t}$ is needed to separate the two actions. This relaxation results in the general policy π shown in (2). With two possible actions and m predicted features, $\hat{\mathbf{c}}, \mathbf{t} \in \mathbb{R}^m$.

$$\pi(\hat{\mathbf{c}}, \mathbf{t}) = \begin{cases} \text{action } 1 & \text{if } \hat{\mathbf{c}} \leq \mathbf{t} \\ \text{action } 2 & \text{if } \hat{\mathbf{c}} > \mathbf{t} \end{cases} \tag{2}$$

The pipeline for using a predictor M_P with such a decision policy is shown in Fig. 2.

Our proposed Decision-Criteria-Informed (DCI) classification model M_{DCI} maps the state $\mathbf{z}$ and decision thresholds $\mathbf{t}$ directly to decisions. The new model pipeline is shown in Fig. 3. This model replaces both the prediction of unknown features and the decision function. The DCI model is a classifier directly trained to predict the decision $\pi(\mathbf{c}, \mathbf{t})$ that

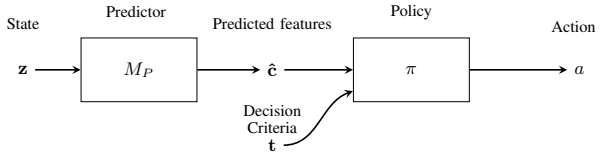


Fig. 2. Predict-then-optimize pipeline: a predictor estimates unknown features, and a policy function π uses those estimates to select an action.

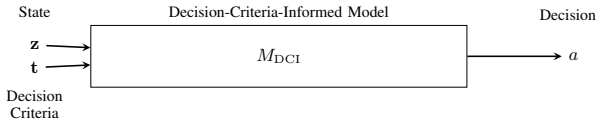


Fig. 3. DCI pipeline: a decision-criteria-informed model maps the state directly to a decision.

would be made by the policy given perfect knowledge of the unknown features $\mathbf{c}$.

IV. Blackjack Experiment

A. Problem Formulation

In this section, we examine our proposed techniques in the game of blackjack. We selected blackjack because it fits the problem class that we wish to address. It is a constrained optimization problem whose goal is to maximize monetary gain at every timestep by choosing from the actions *hit*, *stand*, *split*, or *double*. For simplicity, we use a version of the game with the available actions *hit* and *stand*.

We introduce one additional relaxation to the problem: all outcomes of the same type are considered equally valuable. For that reason, we do not track capital between hands to use as a metric. The same set of hand outcomes can produce different final balances depending on the order in which those outcomes occur. The relaxation also requires us not to grant any additional weight to a "blackjack," or an initial hand of 21. Both of these decisions allow the metrics to gauge the performance of the models more accurately, rather than be influenced by stochastic aspects of the game.

We evaluate the models using the frequencies of the different outcome types. Our primary evaluation metric is the win rate in cases where the game is decisive. This metric is calculated as $\frac{w}{w+l}$, where w is the count of hands the model won during testing, and l is the count of hands lost. The *push* outcome is excluded because it is neutral. We refer to this metric as the *decisive-outcome win rate*.

We frame the simplified blackjack task as a threshold-based decision problem. The goal is to choose an action based on the current player hand p and the final value of the dealer's cards d_{final} at the end of the hand. Because d_{final} is unknown, the decision function uses the predicted final dealer hand $\hat{d}_{\mathrm{final}}$. The policy used to make a decision is shown in (3). The function dictates that the player should take a hit if the dealer is predicted to have a higher-value hand that does not exceed 21, and stand if the dealer is predicted to have a lower-value hand or exceed 21.

$$\pi(\hat{d}_{\mathrm{final}}, p) = \begin{cases} \text{hit} & \text{if } \hat{d}_{\mathrm{final}} > p \text{ and } \hat{d}_{\mathrm{final}} \leq 21 \\ \text{stand} & \text{if } \hat{d}_{\mathrm{final}} \leq p \text{ or } \hat{d}_{\mathrm{final}} > 21 \end{cases} \tag{3}$$

The thresholds used to map the predicted feature $\hat{d}_{\mathrm{final}}$ to an action are the player hand p and the constant value 21. Because the threshold value 21 is a constant determined by the rules of the game, we do not define it as an input for π.

B. Models

We created two models. The first model was a Prediction Focused Learning model. This model is a regressor M_{PFL} that maps the current state $\mathbf{z}$ to the predicted dealer final hand $\hat{d}_{\mathrm{final}}$. The output of this regressor $M_{\mathrm{PFL}}(\mathbf{z}) = \hat{d}_{\mathrm{final}}$ is used in conjunction with the decision policy outlined in (3) to select an action a. This composite function is outlined in (4).

$$a = \pi(M_{\mathrm{PFL}}(\mathbf{z}), p) \tag{4}$$

The second model is our DCI model M_{DCI}. This classifier directly maps the state to actions. It produces hit or stand decisions directly. It was trained to predict the output of the decision function $\pi(d_{\mathrm{final}}, p)$, where p and d_{final} are the true values of the player hand and dealer hand at the end of the hand. It implicitly encapsulates both the task of estimating the dealer's cards at the end of the hand and the task of deciding an action based on that information, as shown in (5).

$$a = M_{\mathrm{DCI}}(\mathbf{z}) \tag{5}$$

C. Methods

We implemented both the PFL model and our proposed DCI model as random forests. Models were trained on all possible timesteps from 75,000 simulated hands of blackjack, resulting in approximately 120,000 total training examples. Each model was then tested on 100,000 simulated hands. Both model types received the following inputs: the player's hand, the dealer's upcard, and a perfect card count from previous hands since the last reshuffle. Each shoe contained four decks and was reshuffled once at least half its cards were dealt. The models received the card values in both hands, as well as bits that signified whether the dealer or player had an ace that was still being counted as 11.

In this experiment, the DCI model M_{DCI} does not receive any additional inputs compared to the PFL prediction model M_{PFL}. The threshold used in the decision policy is the player hand p, which is already included in the input state $\mathbf{z}$ for both models. p is in the input state due to its relevance—along with the card counts from previous hands and all other cards on the table—to calculating the expected value of the cards that may be dealt from the shoe. The difference is that the DCI model predicts an action value conditioned on the player hand, while the PFL model only needs p for its utility in card-value estimation.

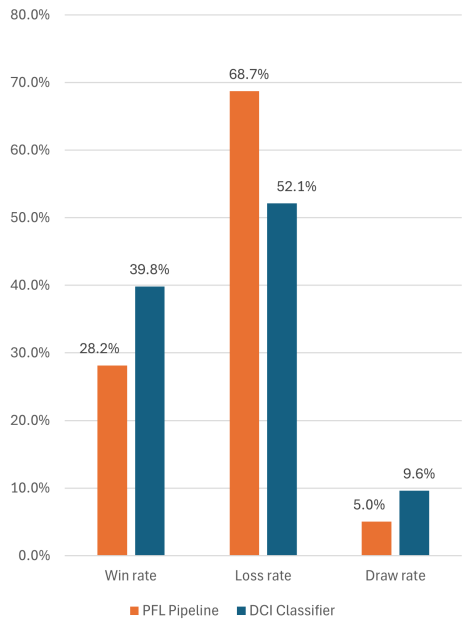


Fig. 4. The outcome rates for the PFL pipeline and the DCI model from the blackjack test hands.

D. Hyperparameter Selection

There were five hyperparameters that needed to be selected for the Random Forest models: maximum tree depth, minimum samples per leaf, maximum-feature selection function at each tree split, number of estimators per forest, and bootstrapping use. These parameters were chosen for both the DCI model and the PFL model independently with three iterations of grid searching. For both models, the grid search selected a max tree depth of 70, a minimum samples per leaf of 1, a square root function for maximum feature count, and no bootstrapping. The classifier performed best with 400 estimators and the regressor was best with 300.

E. Results

The PFL pipeline had a decisive-outcome win rate of 0.291 and the DCI model had a decisive-outcome win rate of 0.433 after the 100,000 test games. The outcome rates for the test hands are shown in Fig. 4. A one-sided two-proportion z-test comparing the models' decisive-outcome win rates found that the DCI model's was statistically significantly higher than the PFL pipeline's ($p < 10^{-16}$).

F. Discussion

In this case, the DCI random forest notably outperformed the PFL decision pipeline. This result occurred even though the DCI model implicitly encapsulates more of the decision-making process and no effort was made to increase its model capacity. These results are promising, but they fail to give a satisfying explanation for the performance gap.

For the next experiment, we wanted to frame a problem in which the model is informed of decision criteria that reflect user choices, rather than an intrinsic aspect of the task as in this case. This allows us to vary the selection of decision criteria during training and observe how this additional input contributes to performance.

The other stipulation for our next experiment was to use neural networks because of their significance in machine learning and because neural-network-based classifiers produce continuous outputs. Continuous outputs can be converted into discrete decisions with different cutoff values, which allows the confidence level required for different decisions to be adjusted. All of these characteristics are reflected in the stock-trading experiment presented in the next section.

V. Stock Market Experiment

A. Problem Formulation

For this paper, we frame stock-market trading as a CO problem where the goal is to decide whether to purchase a given stock at a given timestep. Stocks are purchased at the end of each market day, and the selling period is the next market day from open to close. All purchased stocks have a set take-profit threshold t_{TP} and a stop-loss threshold t_{SL} that can each trigger a sale. If neither threshold is crossed by the end of the market day, the stock is sold at the close price. The decision policy π compares t_{TP} and t_{SL} with $\hat{p}_{\mathrm{high}}$ and $\hat{p}_{\mathrm{low}}$, the market day's predicted highest and lowest price changes relative to the previous day's close. The policy, which is a function of $\hat{\mathbf{p}} = (\hat{p}_{\mathrm{high}}, \hat{p}_{\mathrm{low}})$ and $\mathbf{t} = (t_{\mathrm{TP}}, t_{\mathrm{SL}})$, is detailed in (6). π dictates a purchase if the stop-loss is not predicted to be triggered and the take-profit sale is predicted to be triggered.

$$\pi(\hat{\mathbf{p}}, \mathbf{t}) = \begin{cases} \text{buy} & \text{if } \hat{p}_{\mathrm{high}} \geq t_{\mathrm{TP}} \text{ and } \hat{p}_{\mathrm{low}} > t_{\mathrm{SL}} \\ \text{don't buy} & \text{if } \hat{p}_{\mathrm{high}} < t_{\mathrm{TP}} \text{ or } \hat{p}_{\mathrm{low}} \leq t_{\mathrm{SL}} \end{cases} \tag{6}$$

The state $\mathbf{z}$ that both models use for each stock consists of 50 days of normalized market data containing these features for each day: open price, high price, low price, close price, volume, and day of the week.

B. Models

For this experiment, we created four distinct model classes. All model classes were implemented as LSTMs with a fully connected network.

The first model type is the *PFL prediction model*. Its architecture is shown in Fig. 5. This model receives the state $\mathbf{z}$ as input and predicts two values: the high price and low price for the selling period, each expressed relative to the close price of the previous period. Both outputs are trained with MSE loss. The predictions are used in conjunction with the purchasing policy π to decide an action for a given stock at a given timestep.

The next model type is a fixed-threshold decision classifier that does not receive a threshold input. This model follows the same architecture as the PFL predictor, only with a single sigmoid output in the last layer rather than two linear outputs. This model type was trained to predict when to purchase

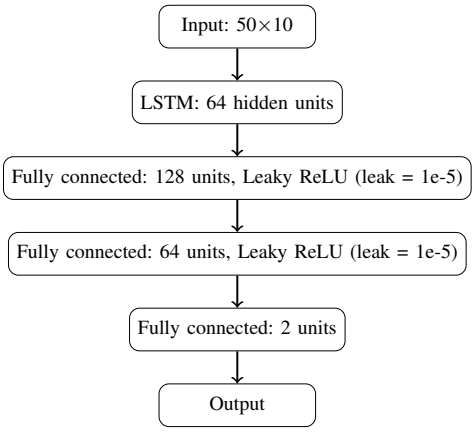


Fig. 5. PFL price predictor model architecture.

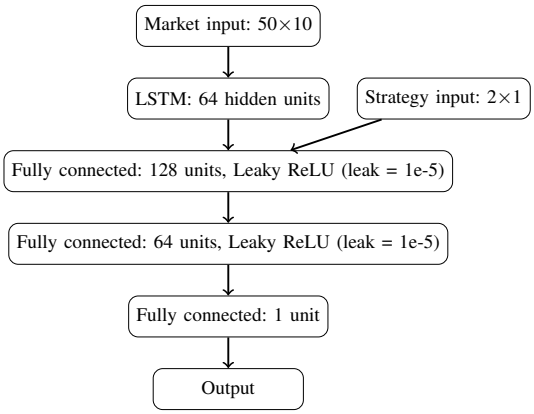


Fig. 6. DCI classifier architecture with selling strategy input.

based on the selling outcome that would occur from using the constant threshold value $\mathbf{t} = (0.02, -0.005)$. In other words, the model identifies stocks which would sell successfully using a 2% take-profit value and a -0.5% stop-loss. Its labels were created by examaning the high and low prices during the trading period. In scenarios where both the stop-loss and take-profit price thresholds are met during the same period,due to the nature of candlestick data, the open price was used to determine the outcome where possible with unresolved outcomes labled as losses. We refer to this model type as the *non-DCI decision classifier*. The purpose of this model is to help gauge how the additional threshold input affects DCI model performance and how much of the increased performance can be attributed only to the models being decision-aware.

The next model type is a *DCI model*. It receives the state $\mathbf{z}$ and an additional selling strategy vector $\mathbf{t} = (t_{\mathrm{TP}}, t_{\mathrm{SL}})$ as input. The additional input vector contains the decision criteria provided to the model. The vector is concatenated to the output of the LSTM, and both are processed together in the fully connected layers. The model outputs a single sigmoid value that represents the probability of a successful sale based on the criteria. The structure of this model is shown in Fig. 6.

During training, each training example was assigned a random threshold input $\mathbf{t} = (t_{\mathrm{TP}}, t_{\mathrm{SL}})$, where t_{TP} was sampled uniformly from $[0.01\%, 10.0\%)$ and t_{SL} was sampled uniformly from $[-5.0\%, -0.01\%)$. The assigned threshold input was used to determine the correct action based on this selling strategy and all price data available from the selling period. All labels were created based on these thresholds in the same manner as the non-DCI decision model.

The final model class we used was a DCI model like the previous one, but with one distinction. This model had all of its training examples assigned a new random threshold input after each epoch of training. Naturally, each label was also updated to be the correct action based on the new threshold vector $\mathbf{t}$. We refer to this model type as the *DCI model with relabeling*.

All classification models in this section produce a continuous output on the interval $(0, 1)$. These models require a function to convert the output to discrete values. This introduces a new hyperparameter: the classification threshold. To avoid confusion with the other thresholds that are referenced in the decision policies, we label this parameter τ. Formally, τ is the threshold in the discretization function that produces the predicted class $\hat{y}$ based on the raw model output $\sigma(z)$, as shown in (7).

$$\hat{y} = \begin{cases} \text{positive} & \text{if } \sigma(z) \geq \tau \\ \text{negative} & \text{if } \sigma(z) < \tau \end{cases} \tag{7}$$

In this case, we define the action "buy" to be the positive class and "don't buy" to be the negative class.

C. Evaluation Metrics

We use two primary metrics to evaluate our models in this section. The first metric is profitability. Profitability is measured by annual return. We identify this as an important metric because return speaks to the real-world utility of a model; however, we do not consider profitability alone a sufficient metric for evaluating model performance. Return in this case is partly a function of the strategy that the model uses to trade. In this paper, it is not our goal to evaluate the usefulness of different stop-loss or take-profit values. Profitability can also vary based on factors besides the trading strategy and the model's skill at identifying instances where that strategy would be successful. For example, two models may both correctly predict that different stocks' prices will at least reach the take-profit price for a given strategy, but one stock may far exceed it before market open and sell at a larger profit. In the given example, profitability would have been a poor metric for identifying how well the models performed at executing that specific strategy.

To adjust for this, we also evaluate models based on their precision and positive rate. Recall that we define a "positive" as a case where the model decides to purchase, and the truth value of each decision is based on whether that purchase would have sold at the take-profit price during the selling period.

We argue that precision is the classification metric that is most relevant. Given the risk associated with stock trading,

a trader generally will care more about the performance of the few stocks they decide to purchase than about the myriad missed opportunities that occur every day. Otherwise, they may decide to purchase every available stock to have perfect recall. This is why we hold precision to be the key classification metric.

However, precision alone is insufficient. It may be clear, for example, that a model with perfect precision that predicts a single 3% increase for a stock once every ten years is not very useful. A certain positive rate is needed for a model to be considered useful. Placing more purchases with the same precision is desirable to increase portfolio diversity and speed up convergence toward the profitability that the model averages; however, we do not focus on this as a need for recall, because it does not pertain to capturing a certain percent of the opportunities available. On the contrary, this is motivated by a desire to have a certain trade volume, regardless of how many opportunities are available. Therefore, we look at the precision-positive rate relationship.

D. Experimental Setup

Eight different datasets were used for training, each spanning a different time period. The first training dataset encompassed the data from 2013–2014, and each subsequent dataset was shifted by one year, ending with 2020–2021. All datasets contained data from the same 500 stocks. Two values were chosen to seed the non-deterministic aspects of the training process. For each model type, an instance was trained using each of the two random seeds on each of the eight datasets, resulting in 16 total model instances per model class. Each model test consisted of one year of data, beginning one year after the end of the training set to avoid data leakage. For example, if a model was trained on data from 2015 through the end of 2016, then it was tested on all the data from 2018.

First, a sweep was conducted to measure the precision of the trained models. All the classification-based model types were swept across different values for the confidence threshold τ. All models were tested with the task of finding stocks that would sell at a 2% increase and not hit a stop-loss set at -0.5%, meaning the strategy was $t_{\text{TP}} = 2\%$, $t_{\text{SL}} = -0.5\%$. This created a direct comparison because the non-DCI decision model is trained only for this strategy. For each classification model, 11 tests were conducted. Tests were conducted for all $\tau \in \{0.1 + 0.08i : 0 \leq i \leq 10\}$. Equivalently, $\tau = 0.1$ was tested, and τ was then incremented by 0.08 for each test until the final value $\tau = 0.9$ was tested.

Next, to test profitability, a sweep was conducted where each trained model was tested with 25 different trading strategy vectors $\mathbf{t}$. The five stop-loss values were selected starting with -0.3%, and were chosen in decrements of 0.3%. The five take-profit values were selected starting with 0.6%, and were chosen in increments of 0.6%. All possible combinations of stop-loss and take-profit values were tested. Formally, given the set of stop-loss values $S = \{-0.3\%(i+1) : 0 \leq i \leq 4\}$, and the set of take-profit values $P = \{0.6\%(i+1) : 0 \leq i \leq 4\}$, a test was conducted for each $\mathbf{t} \in P \times S$. For all these tests, the classification models had their classification threshold values τ set as the constant $\tau = 0.5$.

E. Methods

1) Training: All datasets were created by compiling daily OHLCV values from Yahoo Finance for each stock used [10]. The list of stocks was determined by selecting the 500 stocks with the largest market cap that also had available data throughout 2013–2023. Following common LSTM-based stock-price prediction practice, we trained on min-max normalized price sequences [9]. All input sequences were of length 50 and normalized to the interval $[-1, 1]$. The last 15% of all training examples were selected for the validation set, and then all training examples were shuffled and split into batches of size 200. All model types were trained for 35 epochs with the Adam optimizer. The PFL predictor used a learning rate of 1×10^{-4}, while all classification models were trained with a learning rate of 1×10^{-3}.

2) Testing: For testing, each model type was given a starting balance to trade with. All the test data were then stepped through one day at a time. At each timestep, the model received input and performed inference. The output of each model was interpreted and used to decide whether to purchase a given stock. At each timestep, the model's balance was split evenly among all stocks it selected to purchase. All purchases were filled at the close price for the market day. The next day's data were used to determine what the payoff would be based on which sale orders would be triggered.

The historical data consisted of candlesticks that each represented a full market day. This introduced ambiguity in $\sim 6\%$ of trades placed during the profitability study where both the take-profit price and the stop-loss price were reached during the day. In such cases, we could not identify which price was reached first given the granularity of the data. We decided to assume the stop-loss occurred first in such cases. This assumption can affect profitability estimates because the realized sale price depends on which order was triggered first. It does not affect the calculation of precision in the same way, because such cases are still failures of the model to select purchases that would not reach the stop-loss price during the selling period, and thus should be treated as false positives.

F. Precision Test Results

For each classification boundary τ, all classification-based model types had their precision and positive rates averaged together across all 16 model instances. Since the PFL model was a regression model and therefore has no classification threshold τ, the results from the 16 PFL models were averaged to produce a single point. All of these results were plotted in Fig. 7, with interpolation lines drawn to illustrate trends.

Discussion: The results from this experiment paint a clearer picture of model performance. First, we analyze the precision difference illustrated in Fig. 7. The PFL decision pipeline (orange) shows poor precision at a very low positive rate. The non-DCI decision classifier (light blue) shows limited improvement over PFL at similar positive rates. Recall that

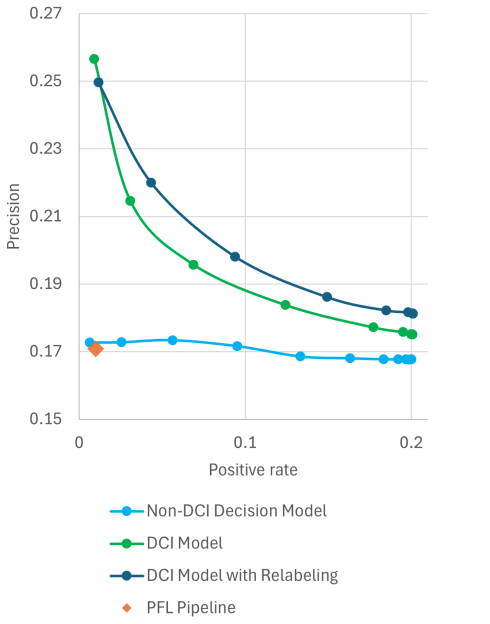


Fig. 7. The average precision and positive rate for each model type. The classification-based models have a point for each τ value tested. Interpolation lines are drawn between points to suggest the full curve for the model classes.

unlike the PFL solution, these models are not decision agnostic. This implies that decision awareness accounts for only a small portion of the performance gap between PFL and the DCI models.

Note the flatness of the curve for the non-DCI decision model. It appears that a lower positive rate (caused by a higher classification threshold τ) is only weakly associated with higher precision. This suggests that the confidence of these models is poorly calibrated, as a higher τ confidence requirement had a limited impact on precision. Both DCI model classes, on the other hand, have substantially more robust curves. The DCI models (green) show a strong correlation between higher confidence and increased precision. At the highest confidence values, the DCI models and DCI models with relabeling performed substantially better than the PFL solution, despite having identical model capacity and the same number of training epochs. This demonstrates that the decision criteria input is helpful for calibrating model confidence.

Consider the following example as a possible explanation. Suppose the DCI model encounters two training examples that have almost identical high prices during the selling period, but have slightly different decision criteria. The label for one may be 1, or "buy," and the other may be 0, or "don't buy." This type of situation could allow the model to calibrate its confidence in the strength of various signals. This effect is further magnified in the relabeled DCI models, which see the exact same training examples epoch-to-epoch with different decision criteria and likely different labels.

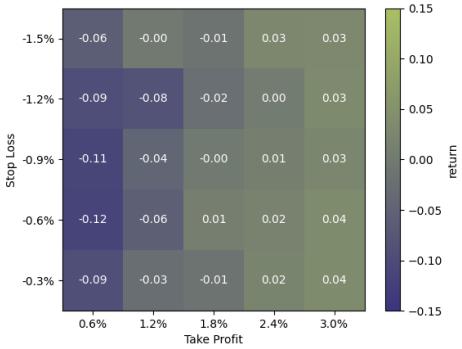


Fig. 8. The average annual return across all model instances for each take-profit and stop-loss value combination tested for the PFL models.

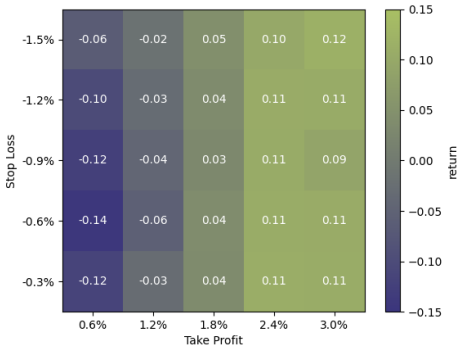


Fig. 9. The average annual return across all model instances for each take-profit and stop-loss value combination tested for the DCI models.

G. Profitability Results

For each unique trading strategy, each model class's annual return was averaged across all 16 model instances. This was done for the PFL models, DCI models, and DCI models with relabeling. The results for the PFL models are shown in Fig. 8. Likewise, the results for the DCI models and the DCI models with relabeling are shown in Figs. 9 and 10, respectively. The average positive rate for all tests across the entire strategy sweep for the PFL models was 4.64%. For the DCI models and the DCI models with relabeling, the averages were 17.04% and 16.90%, respectively.

We compared the average return of the DCI models against that of the PFL models across all strategies using a one-sided Wilcoxon rank-sum test. The test indicated a statistically significant improvement for the DCI models over the PFL baseline ($p \approx 0.001$). A similar test comparing the DCI models with relabeling against the PFL baseline also found a statistically significant improvement ($p \approx 0.001$). A two-sided Wilcoxon rank-sum test between the DCI models and the DCI models with relabeling revealed no statistically significant difference ($p \approx 0.92$).

Discussion: These figures show that the PFL model return values for the best-performing strategies are at times up to

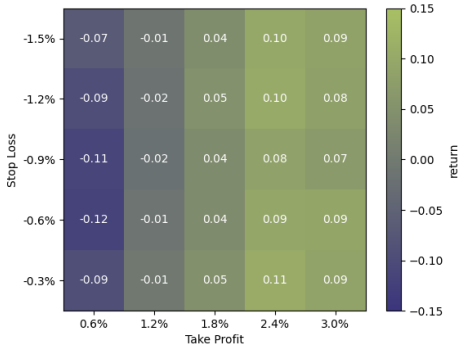


Fig. 10. The average annual return across all model instances for each take-profit and stop-loss value combination tested for the DCI models with relabeling.

11% lower than the DCI model values in Fig. 9. The DCI models achieved this higher level of profitability while having an average positive rate more than three times that of the PFL models. Recall that these model types were identically sized and trained for the same number of epochs. Profitability appears to correlate with the models' respective precisions in this experiment. Despite their higher precision, the DCI models with relabeling show in Fig. 10 similar or lower profitability than the vanilla DCI models. Even so, they still performed notably better than the PFL baseline.

The charts also show that both the DCI and the DCI relabeled models performed worse than the PFL models when using some of the less profitable strategies. We argue that this is not unexpected. Models that have poor precision can be expected to perform relatively similarly across different testing conditions because they do not perform any strategy particularly well. By contrast, a model that has better precision would be expected to have more variation in profitability across different strategies. This follows if the model with better precision executes the strategies more effectively and some strategies are unprofitable.

VI. Conclusion

In this paper, we examined a specific class of predict-then-optimize problems in which decisions are determined by applying thresholds to predicted features. In these settings, a good predictor is not necessarily a good decider: minimizing prediction error and making correct threshold-based decisions are different objectives, and optimizing for one does not guarantee performance on the other. We proposed DCI classifiers as a simpler alternative to existing DFL approaches, replacing the prediction model and threshold policy with a single classifier that receives the decision criteria as direct inputs.

The two experiments together tell a coherent story. DCI is shown to outperform PFL in blackjack but the source of improvement is opaque. The stock-trading experiment and ablation provide deeper explanation. Criteria-awareness, not decision-awareness alone, drives performance, and it does so by calibrating model confidence in a way that prediction-focused pipelines don't replicate. The relabeling results add a nuance: amplifying this calibration effect improves precision but does not consistently translate to better decisions downstream, suggesting the relationship between confidence calibration and task performance warrants further investigation.

We acknowledge certain limitations of this work. The blackjack game used to test the models was a simplified version of the game, and the results it produced may not reflect how the tested models would perform on a non-relaxed version of the game. The stock-market profitability data included a small number of ambiguous outcomes, so it may not be fully reliable. However, the precision results do not suffer the same limitation, and the results in that area corroborate that DCI models outperform their baseline counterparts.

A. Future Work

Future work should explore DCI models further, as their limits have not been fully tested. This technique should be tested in more distinct scenarios, with different baseline models, and with explicit comparisons to other proposed DFL-related solutions. Future experiments should also examine cases where the decision criteria are higher-dimensional or where more complex decision policies can be conditioned on explicit decision criteria.

References

[1] J. Mandi, J. Kotary, S. Berden, M. Mulamba, V. Bucarey, T. Guns, and F. Fioretto, "Decision-focused learning: Foundations, state of the art, benchmark and future opportunities," *arXiv preprint arXiv:2307.13565*, 2023.

[2] M. Qi and Z.-J. Shen, "Integrating prediction/estimation and optimization with applications in operations management," in *Tutorials in Operations Research: Emerging and Impactful Topics in Operations*, INFORMS, 2022, pp. 36–58, doi: 10.1287/educ.2022.0249.

[3] P. L. Donti, B. Amos, and J. Z. Kolter, "Task-based end-to-end model learning in stochastic optimization," in *Advances in Neural Information Processing Systems*, pp. 5484–5494, 2017, doi: 10.48550/arXiv.1703.04529.

[4] A. N. Elmachtoub and P. Grigas, "Smart 'predict, then optimize'," *Management Science*, vol. 68, no. 1, pp. 9–26, Jan. 2022, doi: 10.1287/mnsc.2020.3922.

[5] S. Shah, K. Wang, B. Wilder, A. Perrault, and M. Tambe, "Decision-focused learning without decision-making: Learning locally optimized decision losses," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[6] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for end-to-end planning and control," in *Advances in Neural Information Processing Systems*, 2018, doi: 10.48550/arXiv.1810.13400.

[7] J. Kotary, F. Fioretto, P. Van Hentenryck, and B. Wilder, "End-to-end constrained optimization learning: A survey," in *Proc. 30th Int. Joint Conf. Artificial Intelligence*, 2021, pp. 4475–4482, doi: 10.24963/ijcai.2021/610.

[8] R. Cristian, P. Harsha, G. Perakis, B. L. Quanz, and I. Spantidakis, "End-to-end learning for optimization via constraint-enforcing approximators," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 6, pp. 7253–7260, 2023, doi: 10.1609/aaai.v37i6.25884.

[9] S. Mehtab, J. Sen, and A. Dutta, "Stock price prediction using machine learning and LSTM-based deep learning models," in *Proc. 2nd Symp. Machine Learning and Metaheuristic Algorithms and Applications*, 2020, doi: 10.48550/arXiv.2009.10819.

[10] Yahoo Finance, "Historical market data," Yahoo. [Online]. Available: https://finance.yahoo.com/. [Accessed: May 26, 2026].